

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Miniciclo 1:

- Modificación del tamaño de creación de canvas
- Modificar el método getCanvas
- Modificación del método constructor de circle y rectangle
- setPosition(int x, int y) en Rectangle y en Circle
- Creación de HashMap con estándar CSS

Justificación: Se debe modificar el paquete canvas para poder hacer que el canvas tenga un tamaño más grande y que por ende sea más visible la máquina, también unos métodos que permitan modificar la posición de objetos Rectangle y Circle de manera más simple y no por movimientos continuos, también se debe modificar el método constructor de Rectangle y Circle porque los constructores originales asignaban colores automáticamente, lo cual no sirve como requisito de funcionalidad, también se debe crear un Hashmap para poder cumplir con el requisito de colores de CSS.

Miniciclo 2:

- Symbol(String color, int x, int y)
- getColor()
- makeVisible()
- makeInvisible()
- isSameSymbol(Symbol sym2)
- setPosition(int x, int y)
- isVisible()

Justificación: En este miniciclo se construye toda la clase Symbol ya que son pocos los métodos que hay que definir y son poco complejos.

Miniciclo 3:

- Wheel(int x, int y)
- getCurrentSymbol()
- makeVisible()
- makeInvisible()
- updateSymbol()

Justificación: Se seleccionaron estos métodos para este miniciclo ya que algunos de estos tienen dependencias entre sí para poder funcionar en su totalidad.

Miniciclo 4:

- addSymbol(int pos, String color)
- removeSymbol(String color)
- placeSymbol(String color)
- spin()

Justificación: Se seleccionaron estos métodos para este miniciclo ya que eran los que más trabajaban con la clase Symbol y se podían plantear de forma similar sin perder el hilo en tanto al requisito de que las posiciones se enumeran a partir de 1.

Miniciclo 5:

- SlotMachine()
- addWheel(pos)
- delWheel(pos)
- addSymbol(pos, color)
- delSymbol(symbol)

Justificación: Sin ruedas ni símbolos no se puede ver ni probar nada, también los agrupe por que comparten el requisito de que las posiciones se enumeran a partir de 1 y de propagar los cambios a las otras clases.

Miniciclo 6:

- placeSymbol(wheel, symbol)
- spin()
- spin(wheel)
- isJackpot()

Justificación: Son los que definen lo que muestra cada rueda y los que verifican si hay jackpot, así que estos van juntos por que sin ellos jackpot no tendría nada que evaluar.

Miniciclo 7:

- Symbols()
- configuration()
- distinctSymbols()
- makeVisible()
- makeInvisible()
- exit()
- ok()
- showError(message)

Justificación: Se seleccionaron estos métodos para este miniciclo ya que son los que leen o muestran el estado de la máquina con avisos visuales al usuario y no generan modificaciones lógicas sobre ella.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

Ya se terminaron todos los miniciclos planteados

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

17 Horas / Samuel

10 Horas / Sofía

4. ¿Cuál considera fue el mayor logro? ¿Por qué?

El mayor logro fue conseguir la mayoría de funcionalidades propuestas en el ciclo con pocos bugs de interfaz que no pudimos solucionar.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue de dibujo de canvas ya que al terminar el código, la lógica de dibujado funcionaba, pero como en el canvas los colores se dibujan encima de otras al hacer cambios sobre la máquina como al actualizar el estado de jackpot a true, se dibujaba la máquina encima de todo lo otro o casos en los que las ruedas se dibujan encima de los símbolos, así que para resolver eso tuvimos que volver a revisar el código desde wheels para realizar cambios en el orden de algunas líneas y hacer prueba y error hasta que se dibujó todo en el orden necesario.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Lo que hicimos bien como equipo fue distribuir el trabajo cuando nos dimos cuenta que no alcanzamos si hacíamos todo al mismo tiempo, para mejorar los resultados nos comprometemos a dedicar más tiempo al proyecto y organizarnos mejor.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿Cuál fue la más útil? ¿por qué?

El trabajo por módulos, ya que se hizo progresivamente de lo más complicado a lo mas difícil, comenzando por modificar los métodos canvas, después crear la clase symbol, la clase wheel y por último la clase SlotMachine.

8.¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Se utilizó Claude – Anthropic. (2026). *Claude* (Sonnet 5) [Modelo de lenguaje de gran escala]. <https://claude.ai> – como asistente para estructurar el código de mapeo de colores CSS en la clase Canvas, y como referencia de consulta el estándar CSS Color Module Level 4 del W3C para obtener los 147 nombres de colores estándar y sus valores hexadecimales correspondientes – W3C. (2022). *CSS Color Module Level 4*. World Wide Web Consortium. <https://www.w3.org/TR/css-color-4/> – .

Retrospectiva Ciclo 2:

1.¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Miniciclo 1:

`swap(wheel1, wheel2)`

Justificación: Se tomó este método como primer miniclip ya que es la funcionalidad que consideramos más fácil de implementar entre las 4 nuevas.

Miniciclo 2:

`lock(wheel)`

`unlock(wheel)`

`getLocked()`

`setLocked()` en Wheel

Justificación: Se agruparon porque estos son necesarios para manipular completamente el estado locked, además de esto se debieron agregar las validaciones de bloqueo que debían agregarse a los métodos ya existentes de ciclo 1, tomando decisiones de diseño de cómo se iba a comportar cada uno de estos con respecto al nuevo estado posible de las ruedas, los métodos modificados ya existentes fueron (`placeSymbol(String : color)`, `spin()`, `spin(wheel)`).

Miniciclo 3:

`spin(wheel, steps)` en SlotMachine

`spin(steps)` en Wheel

Justificación: Se trabajaron juntos porque funcionan tipo cascada ya que en SlotMachine se hacen las validaciones y en Wheel se ejecuta toda la lógica para hacer que el índice de Wheel se desplace de forma circular sobre los símbolos según la cantidad dada de pasos.

Miniciclo 4:

`spin(setSymbols[])` en SlotMachine

Justificación: Se dejó de últimas porque es la funcionalidad más completa y compleja, ya que tiene bastantes validaciones, entre ellas verificar que los colores pertenezcan a la lista de `masterSymbols`, que el tamaño de símbolos no sea mayor a la cantidad de símbolos en `masterSymbols` etc.

2.¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?
Ya se terminaron todos los miniciclos planteados.

3.¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)
13 Horas / Samuel
4 Horas / Sofía

4.¿Cuál considera fue el mayor logro? ¿Por qué?
El mayor logro fue identificar y corregir bugs del ciclo 1 que no habíamos podido identificar en ese entonces por la falta de conocimiento sobre el manejo de pruebas unitarias.

5.¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
Gracias al manejo de las pruebas unitarias en este ciclo sobre todo en la función lock que decidimos ejecutar pruebas que incluyen asserts sobre coincidencia de colores, identificamos errores en Wheel : addSymbol donde no se estaba ajustando el índice al insertar un nuevo símbolo lo que podría generar un cambio de símbolo dentro de la rueda sin que el usuario lo hubiera pedido y generaba errores de assert.
La solución fue incrementar index cuando la posición de inserción es menor o igual al índice actual protegido con la variable empty para no afectar el caso de la primera inserción de una rueda vacía en donde el índice debe mantenerse en 0.

identificamos errores en SlotMachine : isJackpot donde no había protección contra las ruedas sin símbolos, ya que dentro de este se llamaba getCurrentSymbol() en cada rueda sin verificar que tuviera símbolos, lo que podía producir IndexOutOfBoundsException.
La solución fue agregar un condicional que retornará en caso de que la rueda inicial o alguna rueda dentro del ciclo no tenga símbolos.

identificamos un error en SlotMachine : updateJackpotStatus() que forzaba la visibilidad de la rueda incondicionalmente, lo cual descubrimos porque en el enunciado se pedía la condición de que las pruebas debían ser ejecutadas en modo Invisible y al ejecutar cierta prueba se hacía todo visible.
La solución fue agregar un condicional que hiciera visible para visibilizar los cambios en caso de que la máquina ya fuera visible.

También hubo algunos problemas en el orden de verificación de condiciones dentro de placeSymbol() y spin(wheel: int), ya que se estaba verificando que no hubieran ruedas bloqueadas antes de verificar si siquiera existían ruedas, la solución fue intercambiar el orden.

6.¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Lo que hicimos bien como equipo fue distribuir el trabajo cuando nos dimos cuenta que no alcanzamos si hacíamos todo al mismo tiempo, para mejorar los resultados nos comprometemos a dedicar más tiempo al proyecto y organizarnos mejor.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿Cuál fue la más útil? ¿por qué?

Lo más útil fueron las pruebas unitarias, ya que permiten hacer seguimiento al código y encontrar bugs fácilmente.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Se utilizó Claude – Anthropic. (2026). *Claude* (Sonnet 5) [Modelo de lenguaje de gran escala]. <https://claude.ai> – como asistente para estructurar el código de mapeo de colores CSS en la clase Canvas, y como referencia de consulta el estándar CSS Color Module Level 4 del W3C para obtener los 147 nombres de colores estándar y sus valores hexadecimales correspondientes – W3C. (2022). *CSS Color Module Level 4*. World Wide Web Consortium. <https://www.w3.org/TR/css-color-4/> – .

Se consultó Junit <https://docs.junit.org/current/user-guide/> como referencia para el uso de `@BeforeEach`, `@AfterEach` y `@Test` en las pruebas unitarias.